

Help Python

Tipos Primitivos:

Int = Inteiro -----> 1, 7, 100, -2, -45

str = string ou Texto -----> Palavras

float = Real -----> 1.4, 98.0, -1.6, 00.5

bool = Verdadeiro e Falso -----> True, False

Operadores Aritméticos:

+	Soma	=	Atribuir
-	Subtração	==	Comparar
/	Divisão	!=	Diferente
*	Multiplicação	%	Resto
**	Potência	** (1/2)	Raiz Quadrada
//	Divisão Inteira	* 0.?	Porcentagem

+= Atribuir a soma de um valor, com um valor já existente.

Código "IS":

```
is + código  
EX: var.isnumeric  
var = variável  
is = ele é  
numeric = código
```

```
isnumeric = numérico  
isalpha = alfabético  
isalnum = numérico e alfabético  
isupper = somente letras maiúsculas  
isdecimal = decimal  
isspace = espaço  
islower = somente letras minúsculas  
isprintable = pode ser impresso
```

Exibição e Formatação:

Ps: Para a formatação acontecer, a variável tem que ser de tipo 'str'

```
print('{:.2f}'.format(código))  
print(f'{variável:.2f}')  
:.2f -----> Arredondamento  
:>2 -----> Alinhado a Direita  
:<2 -----> Alinhado à Esquerda  
:^2 -----> Centralizado  
end="" -----> mesma linha  
\n -----> pula linha  
input() -----> Imputa uma pergunta.  
EX: var = int(input('Pergunta'))
```

Fatiamento de strings:

`variável.count(' ', 0, 0)`

Serve para encontrar elementos de uma string.

`variável.replace(' ', ' ')`

Serve para trocar elementos de uma string.

`variável.upper()`

Serve para manter o que está maiúsculo, e deixa maiúsculo o que não está.

`variável.lower()`

Serve para manter o que está minúsculo, e deixa minúsculo o que não está.

`variável.capitalize()`

Serve para deixar todos os 1º elementos da string minúsculos.

`variável.title()`

Serve para deixar todos os 1º elementos da string maiúsculos.

`variável.strip()`

Serve para remover espaços inúteis no começo e no fim de uma string.

`variável.rstrip()`

Serve para remover espaços inúteis apenas no fim de uma string.

`variável.lstrip()`

Serve para remover espaços inúteis apenas no começo de uma string.

`variável.split()`

Serve para dividir a string nos espaços, gerando novas strings.

`".join(variável)`

Serve para juntar strings, gerando uma nova string.

Cores no Terminal:

`\033[Estilo;Texto;Fundo`

EX: `\033[1;33;44m`

Estilo:	Texto:	Fundo:
0 = nada	30 = Preto	40 = Preto
1 = negrito	31 = Vermelho	41 = Vermelho
4 = sublinhado	32 = Verde	42 = Verde
7 = inverter	33 = Amarelo	43 = Amarelo
	34 = Azul	44 = Azul
	35 = Magenta	45 = Magenta
	36 = Ciano	46 = Ciano
	37 = Cinza	47 = Cinza

Importação de Bibliotecas:

Serve para importar uma biblioteca.

EX: -----> import biblioteca

EX: -----> from biblioteca import Específico

PS: Caso queira dar um apelido a biblioteca. Usar:

EX: -----> import biblioteca as apelido

Random = Aleatório shuffle: -----> embaralha de forma aleatória randint: -----> Randomiza números inteiros	Pyautogui = cria um bot click() -----> Clicar press() -----> Pressionar uma tecla write() -----> escrever PAUSE = -----> Pausa entre códigos hotkey() -----> Atalho position() -----> Descobrir posição do mouse scroll() -----> Rolar pagina para cima
pygame = jogos, vídeos, musicas	datetime = datas date: -----> Informa data atual
time = Tempo sleep -----> Tempo no programa	math = Matemática

Condicional Simples e Composta:

Funções: if, elif, else

If -----> se EX: if variável1 >= variável2: print(variável1) EX: if variável.isnumeric: print(variável1)	EX Principal: if variável condição variável: EX: if variável in lista: print(variável1) EX: if 0 <= variável <= 10: print(variável1)
elif -----> senao-se EX: if variável1 >= variável2: print(variável1) EX: if variável.isnumeric: print(variável1)	EX Principal: elif variável condição variável: EX: if variável in lista: print(variável1) EX: if 0 <= variável <= 10: print(variável1)
else -----> senao EX Principal: else: EX: else: print('Resultado')	

Definições:

if	elif	else
Avalia uma condição. Se for verdadeira, o bloco de código associado será executado.	(abreviação de else if): Define condições adicionais que são verificadas apenas se a condição anterior (if ou outro elif) for falsa.	Define um bloco de código que será executado caso todas as condições anteriores sejam falsas.

Estrutura de Repetição:

Funções: for, while, while True

For -----> Para EX: for c in range(0, 10): print('Repetir 10 vezes') EX: for c in range(var1, var2): print('começa em var1 termina em var2')	EX Principal: for variável C in comando(controle): EX: for c in lista(): print('Repetir até os elementos acabarem') EX: for c, v in lista(): print('para listas com muitos elementos')
while -----> Enquanto EX: while cont < num: print('enquanto cont for menor que num') EX: while num not in 'SN': print('enquanto num não for S ou N')	EX Principal: while variável condição variavel2: EX: while cont != 0: print('enquanto cont for diferente de 0')
While True -----> Enquanto Infinito EX: while True: break para quando esta condição for atingida.	EX Principal: while True: EX: while True: continue para quando esta condição for atingida.

Definições:

for	while	while True
É usado para iterar sobre uma sequência (como listas, tuplas, strings ou intervalos). O loop executa o bloco de código uma vez para cada item da sequência.	Repete o bloco de código enquanto uma condição for verdadeira.	Cria um loop infinito, que só será interrompido manualmente com break, continue ou outros mecanismos.

Funções:

def -----> definir função

def no Python é usada para definir funções, que são blocos de código reutilizáveis projetados para executar uma tarefa específica. As funções permitem organizar melhor o código, reduzir repetição e melhorar a legibilidade.

Componentes principais:

def: Inicia a definição da função.

Nome da função: Deve ser descritivo e seguir as convenções de nomenclatura (ex.: minha_funcao).

Parâmetros (opcional): Variáveis que podem ser passadas para a função, entre parênteses.

Bloco de código: O que a função fará quando chamada.

return (opcional): Especifica o valor que a função deve devolver após a execução.

EX: def nome(parâmetros): P = parâmetro + 1 return p	EX: def nome(*parâmetros): * = para receber vários parâmetros	EX: def nome(*parâmetro=0): quando um parâmetro não for inicializado ele, a def utilizará o que foi declarado.
--	--	---

Variáveis Compostas:

Funções: Tuplas = (), Listas = [], Dicionários = {}

Funções que poderão ser utilizadas em todas as Variáveis Compostas:

max(variável) = Maior.

min(variável) = Menor.

variável.index = Saber o índice de um elemento.

variável.insert(posição, item) = Adiciona um item em um ponto específico da lista.

variável.clear() = Apaga todos os valores de uma lista.

len(variável) = Ler a quantidade de elementos.

type(variável) = Ler a variável de um elemento.

Tuplas = ()

Definição: Uma coleção imutável de itens ordenados.

Características:

Após criada, seus valores não podem ser alterados (adicionar, remover ou modificar itens).

variável.sorted = Ordem alfabética.

Listas = []

Definição: Uma coleção mutável de itens ordenados.

Características:

É possível adicionar, remover ou modificar elementos.

variável.append(item) = Adiciona um item à lista.

variável.pop(índice) = Remove o item do índice selecionado.

variável.remove(item) = Remove o item selecionado.

variável.sort() = Ordena os valores em ordem crescente.

variável.sort(reverse = True) = Ordena os valores em ordem decrescente.

[:] = Desvincular duas ou mais listas.

Dicionário = {}

Definição: Uma coleção mutável de pares chave-valor.

Características:

Os valores são acessados pelas chaves (não por índice).

variável.values() = Os valores registrados nas chaves.

variável.keys() = As chaves registradas nos índices.

variável.items() = Tudo! chaves e valores.

variável.copy() = Desvincular duas ou mais listas.

key=itemgetter(índice) = Organizar dicionários em ordem decrescente. porém transforma em uma lista, para utilizar declare uma nova lista.

Tratamento de Erros:

try: --> Tente Fazer Operação ou Programa.	Except: --> Se Ocorreu Erro Faça isso.
else: --> Se Deu Certo Faça isso.	finally: --> Acontece logo após a execução dos comandos, independente se um erro foi ocasionado.

PS: O except: pode ser inicializado várias vezes.

EX: try:

except:

except:

else:

PS: Depois do except: deve ser informado o erro.

EX: except Type Erro:

EX: except (Type Erro, Value Erro):
